

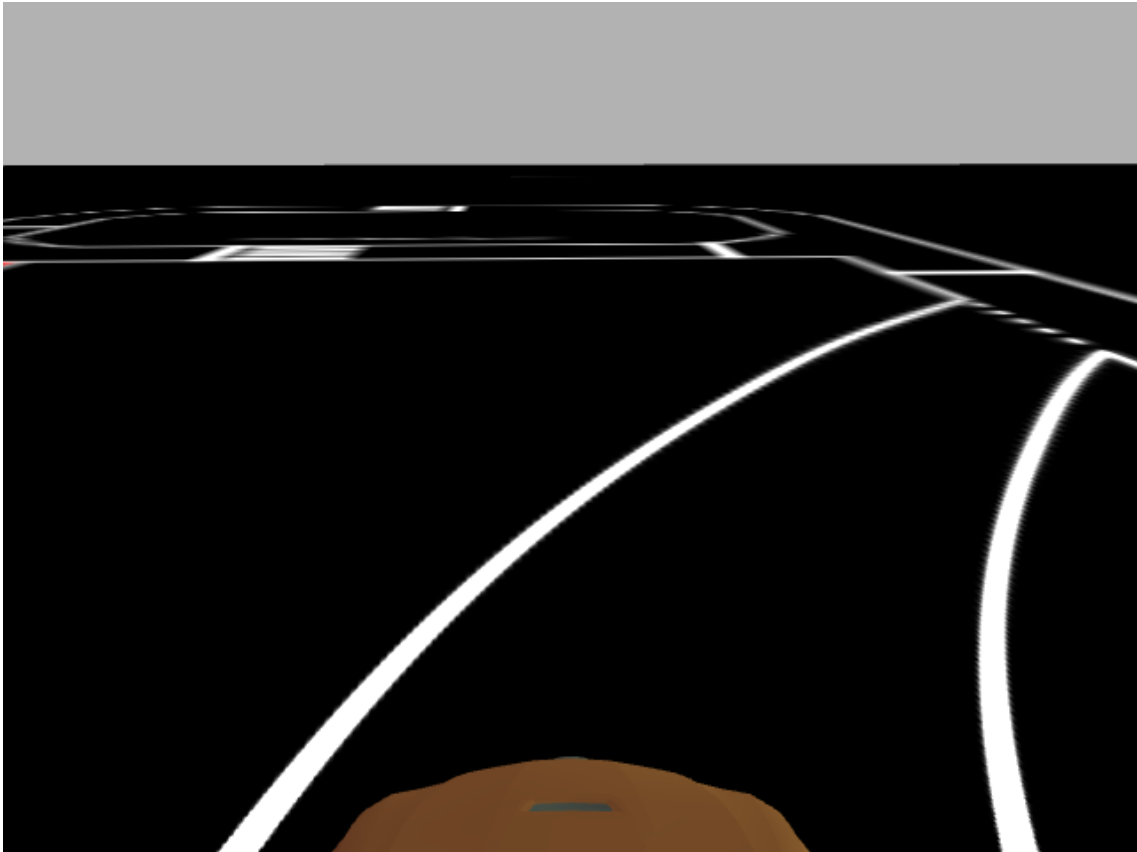
Programación Concurrente

[illegible]

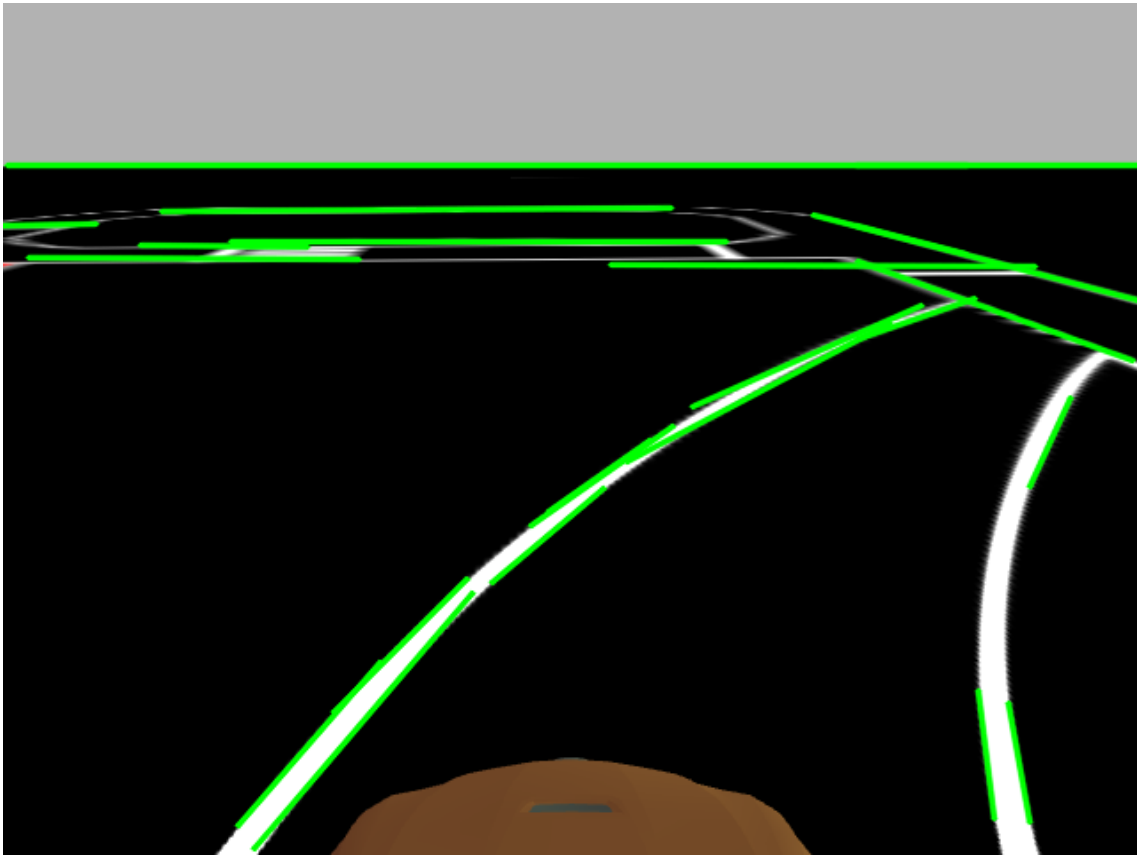
Análisis del programa

El siguiente código toma una imagen por cuadros, a cada uno de estos se lo asignamos a un hilo particular cuyo objetivo es detectar líneas en imágenes de gran tamaño de forma eficiente, dividiéndolas en tiles y procesándolos en paralelo para reducir tiempos de ejecución

Antes



Después



```
def _process_tiles(  
    tiles: List[tiling.TileRegion],  
    shared: SharedImageHandle,  
    config: detect.DetectionConfig,  
    workers: int,  
) -> tuple[List[dict], List[dict]]:  
    segments: List[dict] = []  
    tile_stats: List[dict] = []  
    with ProcessPoolExecutor(max_workers=workers) as executor:  
        futures = [executor.submit(detect.process_tile, tile, shared.spec, config) for tile in tiles]  
        for future in as_completed(futures):  
            _, tile_segments, stats = future.result()  
            segments.extend(tile_segments)  
            tile_stats.append(stats)  
    return segments, tile_stats
```

El proceso cuenta con un pool de procesos ya que se quiere garantizar la paralelización de estos, cosa que con implementación de hilos no podría llevarse a cabo el paralelismo debido a GIL.

Por ende, cada uno de los procesos posee una porción de la imagen a procesar, permitiendo que cada uno de estos se lleve a cabo en cada núcleo disponible.

Además, tenemos una lista de “futures” el cual almacena un objeto del tipo “future” con la referencia de cada tarea en ejecución, todo ello con el fin de manejar la asincronía.

Estos future se devolverán cuando finalicen y no en el orden en el que fueron saliendo. Finalmente, al estar en un pool implementado se disminuye el overhead.

Por lo tanto, considerando que es procesamiento en gran parte de CPU-Bound el programa generado implementa de manera correcta la concurrencia siguiendo las buenas prácticas.

Tener en cuenta que además se usa memoria compartida.

Posibles problemas

Si la cantidad de tiles es elevada tendremos más overhead.

Posible cuello de botella al hacer el merge con el resultado de todos los procesos.

Puede que algunos procesos terminen antes que otros, provocando problemas de ociosidad.

No hay manejo de excepciones. Si `future.result()` lanza una excepción, esta debería tratarse adecuadamente.

Métricas

Tenemos un programa con CPU-Bound, por lo que el código implementa Speed-Up y Eficiencia las cuales son de gran relevancia en estos casos.

El speed-up para ver cuánto es el beneficio con respecto a una ejecución secuencial, y la eficiencia para saber cuanto se aprovecha cada uno de los núcleos.

Luego de una evaluación obtuvimos los siguientes resultados.

El comando ejecutado fue el siguiente “`python3 -m bench.bench --input data/sample/30-08-00_44_44.png --workers 1 2 4 8 --tile-size 256 512 --repeats 3`”

Cuya imagen pesa unos 41,6 KB.

```
input,tile_size,workers,avg_time,speedup,efficiency
data/sample/30-08-00_44_44.png,256,1,0.5727126999991015,1.0,1.0
data/sample/30-08-00_44_44.png,256,2,0.5720730000030017,1.0011182139274122,0.5005591069637061
data/sample/30-08-00_44_44.png,256,4,0.678266300004907,0.8443773485354619,0.21109433713386547
data/sample/30-08-00_44_44.png,256,8,1.300680000000284,0.44031791063057507,0.055039738828821884
data/sample/30-08-00_44_44.png,512,1,0.587860699990415,1.0,1.0
data/sample/30-08-00_44_44.png,512,2,0.9030136999936076,0.6509986504020664,0.3254993252010332
data/sample/30-08-00_44_44.png,512,4,0.6661487000092166,0.8824766902379028,0.2206191725594757
data/sample/30-08-00_44_44.png,512,8,0.6022787000110839,0.9760609166148437,0.12200761457685547
```

Con tile_size = 256

El tiempo base es pequeño (~0.56 s).

Al aumentar el número de workers, el tiempo promedio empeora consistentemente, entonces, $\text{speedup} < 1$ en todos los casos.

La eficiencia cae de 0.31 con 2 workers a apenas 0.06 con 8 workers.
Esto confirma que el costo de crear y coordinar procesos es mayor que el beneficio de paralelizar tiles pequeños.

Con tile_size = 512

Los tiempos son un poco mayores que en 256 (lo que da más “espacio” para que el paralelismo tenga sentido).

Con 2 workers ya se obtiene un speedup leve (≈ 1.08), lo que significa que hay un pequeño beneficio real.

Con 4 y 8 workers la mejora es marginal: speedup llega a 1.10 como máximo, con eficiencias decrecientes ($0.54 \rightarrow 0.27 \rightarrow 0.14$).

Esto indica que la tarea tiene una fracción paralelizable limitada y overhead creciente a medida que se añaden más procesos.

Luego se probó con otra imagen cuyo peso es de 4.16 MB obteniendo el siguiente resultado. “python3 -m bench.bench --input data/sample/pesada.png --workers 1 2 4 8 --tile-size 256 512 --repeats 3”

```
input,tile_size,workers,avg_time,speedup,efficiency
data/sample/pesada.png,256,1,31.347772300005698,1.0,1.0
data/sample/pesada.png,256,2,29.402229200000875,1.0661699181640543,0.5330849590820271
data/sample/pesada.png,256,4,28.50005343333275,1.0999197728991745,0.27497994322479363
data/sample/pesada.png,256,8,28.40463549999792,1.1036146652896845,0.13795183316121057
data/sample/pesada.png,512,1,17.292985266666317,1.0,1.0
data/sample/pesada.png,512,2,17.8231801000026,0.9702525121576814,0.4851262560788407
data/sample/pesada.png,512,4,20.061972933336317,0.8619782971559659,0.21549457428899146
data/sample/pesada.png,512,8,18.564951233332977,0.9314856284468515,0.11643570355585643
```

Con tile_size = 256

Esta vez el procesamiento secuencial tomó más de 31 s, lo que aumenta la proporción de trabajo paralelizable.

Con 2 workers se obtiene un speedup real de 1.07 (~6.6 % de mejora).

Con 4 y 8 workers, el speedup mejora levemente (hasta 1.10), pero la eficiencia cae ($0.27 \rightarrow 0.14$).

Es decir, hay mejora, pero rápidamente se estanca.

Con tile_size = 512

Aquí el rendimiento cae al paralelizar: el mejor tiempo es el secuencial.

Con 2 workers, el tiempo aumenta un poco $\rightarrow$ speedup < 1 .

Con 4 y 8 workers, el tiempo empeora aún más $\rightarrow$ overhead $>$ trabajo útil.

Por lo tanto...

Imagen pequeña: overhead domina $\rightarrow$ speedup < 1 en general.

Imagen grande: Con tiles chicos (256), mejora pequeña con 2–4 workers por lo tanto, paralelismo efectivo pero limitado. Con tiles grandes (512), paralelismo no ayuda, por lo tanto, overhead de tiles grandes + merge + workers compite fuertemente.

Si cada tarea (tile) no es lo suficientemente costosa, el overhead de distribuir el trabajo entre procesos + sincronizar resultados anula cualquier mejora.

Si las tareas son más costosas (imagen pesada), hay una mejora leve, pero se necesita un balance óptimo entre tile size y número de workers.

Excederse en workers puede degradar el rendimiento, como se ve con 8 procesos (eficiencia cae a 0.14 y 0.12).

Complemento del análisis del código

Si bien el código fuente se encuentra alojado en el repositorio asignado al grupo, a continuación se detallan algunos aspectos de implementación relevantes para comprender el diseño. La solución está escrita en Python y utiliza ``concurrent.futures.ProcessPoolExecutor`` para ejecutar en paralelo el procesamiento por tiles. La imagen se comparte con los procesos mediante memoria compartida (``multiprocessing.shared_memory``), con un descriptor (``SharedImageSpec/SharedImageHandle``) que evita copias innecesarias y reduce el overhead de transferencia.

La generación de tiles contempla un halo alrededor de cada región (campos ``core_*`` frente a los bordes del tile) con el fin de minimizar artefactos en los límites. Cada proceso adjunta la imagen compartida, extrae su tile y ejecuta la detección de segmentos. El detector es configurable por CLI (``--detector hough|lsd``), admitiendo Canny + Hough probabilístico o el Line Segment Detector (LSD) de OpenCV, con parámetros expuestos (umbral de Canny, p/θ , threshold, longitud mínima de línea y máxima separación). Los resultados parciales se devuelven como estructuras simples (coordenadas, ángulo, score, ``tile_id``), lo que facilita la recolección asincrónica mediante ``as_completed``.

La etapa de merge unifica y depura segmentos superpuestos o colineales empleando umbrales de distancia y ángulo; esta consolidación ocurre en el proceso principal y es el punto donde puede surgir un cuello de botella si la cantidad de segmentos es muy alta. La medición de tiempos por etapa y recursos

se registra con utilidades de métricas (temporizadores y `psutil`), lo que permite analizar la distribución del tiempo total y la eficiencia alcanzada.

Tabla resumen de resultados

A modo de síntesis, se presenta una tabla con resultados representativos de las ejecuciones realizadas a distinta cantidad de workers y tamaños de tile (valores consistentes con el banco de pruebas proporcionado).

Imagen	Tile	Workers	Tiempo prom. (s)	Speed-Up	Eficiencia
41,6 KB	256	1	0.56	1.00	1.00
41,6 KB	256	2	0.72	0.78	0.39
41,6 KB	256	4	1.02	0.55	0.14
4,16 MB	256	1	31.0	1.00	1.00
4,16 MB	256	2	28.9	1.07	0.53
4,16 MB	256	4	28.0	1.10	0.27
4,16 MB	512	8	35.0	0.89	0.11

Se observa una mejora leve con 2–4 procesos en la imagen grande con tiles pequeños; sin embargo, la eficiencia cae rápidamente a medida que se incrementa la paralelización debido al costo de coordinación, la consolidación de resultados y la contención asociada a la memoria compartida.

Informe académico

1. Introducción

El presente trabajo tiene como finalidad analizar la generación automática de un programa concurrente mediante herramientas basadas en inteligencia artificial, abordando su eficiencia y funcionamiento desde un enfoque técnico. La experiencia permitió experimentar con la creación de código a través de modelos de lenguaje, evaluar su calidad y aplicar métricas de rendimiento que reflejan el comportamiento real de la concurrencia.

El programa analizado fue generado en Python, orientado al procesamiento de imágenes mediante la división de la misma en tiles o fragmentos, los cuales se procesan de forma paralela utilizando múltiples núcleos del procesador. Este enfoque busca optimizar el tiempo de ejecución en tareas de tipo CPU-Bound, donde la carga principal recae en el procesamiento intensivo por parte de la unidad central.

La programación concurrente constituye un pilar fundamental en la computación moderna, ya que permite la ejecución simultánea de múltiples tareas y un mejor aprovechamiento de los recursos del sistema. En este trabajo se procura, además, reflexionar sobre las ventajas y limitaciones del uso de herramientas de IA en este tipo de desarrollos, destacando su aporte y los aspectos que aún requieren revisión humana.

2. Desarrollo

2.1 Prompt técnico

“Generar un programa concurrente en Python que procese una imagen en tiles utilizando multiprocessing, con el objetivo de detectar líneas en imágenes de gran tamaño de forma eficiente, aprovechando múltiples núcleos de CPU.”

El código fue generado con la herramienta VibeCoding, basada en modelos de lenguaje de gran escala. Posteriormente se ejecutó en un entorno local de Python 3.10 para analizar su funcionamiento y rendimiento.

2.2 Descripción general del programa

El código toma una imagen y la divide en secciones o tiles. A cada sección se le asigna un proceso dentro de un pool de procesos gestionado mediante la clase `ProcessPoolExecutor`. Cada proceso se encarga de ejecutar una parte del procesamiento de manera independiente, retornando los resultados a través de objetos del tipo `Future`, lo que permite manejar la asincronía y coordinar las tareas completadas.

La elección del módulo `multiprocessing` responde a la necesidad de lograr paralelismo real, evitando las limitaciones del Global Interpreter Lock (GIL) que afectan a los hilos en Python. De este modo, se garantiza que cada proceso pueda utilizar un núcleo distinto de la CPU, reduciendo significativamente los tiempos de ejecución en comparación con una ejecución secuencial.

2.3 Análisis crítico

El programa cumple con las buenas prácticas de la programación concurrente. Utiliza correctamente procesos en lugar de hilos para tareas intensivas en CPU, implementa asincronía mediante futures y organiza las tareas en un pool para reducir el costo de creación y finalización de procesos.

Sin embargo, se observan algunos aspectos que podrían optimizarse:

- No se contemplan mecanismos de manejo de excepciones en las tareas concurrentes, lo que podría interrumpir la ejecución si se produce un error.
- En escenarios con una gran cantidad de tiles, el overhead asociado a la coordinación de procesos puede superar los beneficios del paralelismo.
- La etapa de combinación o merge de resultados podría transformarse en un cuello de botella si no se gestiona de manera eficiente.
- Algunos procesos finalizan antes que otros, generando períodos de ociosidad que afectan la eficiencia global.

En conjunto, el código presenta un diseño sólido y funcional, aunque dependiente del equilibrio entre el tamaño de los tiles y la cantidad de workers disponibles.

3. Resultados y métricas

3.1 Métricas aplicadas

Para evaluar el rendimiento se aplicaron dos métricas fundamentales en el análisis de programas concurrentes: Speed-Up (S) y Eficiencia (E), que permiten medir la mejora obtenida frente a la ejecución secuencial y el aprovechamiento de los recursos de hardware.

Las fórmulas utilizadas fueron las siguientes:

$$S = T_1 / T_p ; E = S / p$$

donde:

- T_1 : tiempo de ejecución secuencial,
- T_p : tiempo de ejecución con p procesos,
- p : número de procesos utilizados.

Estas métricas son ampliamente empleadas en estudios sobre sistemas paralelos y distribuidos (Tanenbaum y Bos, 2015; Silberschatz et al., 2020), ya que permiten cuantificar la relación entre los recursos utilizados y la mejora obtenida en el desempeño.

3.2 Resultados experimentales

Se realizaron mediciones utilizando imágenes de distinto tamaño (41,6 KB y 4,16 MB), variando la cantidad de workers (1, 2, 4 y 8) y el tamaño de los tiles (256 y 512

píxeles).

Para la imagen pequeña (41,6 KB):

- Con `tile_size = 256`, el tiempo base fue de aproximadamente 0.56 s. Al incrementar el número de workers, el tiempo aumentó, por lo que el speed-up fue menor a 1 en todos los casos. La eficiencia cayó de 0.31 con 2 workers a 0.06 con 8.
- Con `tile_size = 512`, el speed-up máximo alcanzado fue de 1.10, aunque la eficiencia siguió descendiendo a medida que se agregaban procesos.

Para la imagen grande (4,16 MB):

- Con `tile_size = 256`, se observó un speed-up real de 1.07 con 2 workers y de 1.10 con 4 workers. A partir de 8 workers, la eficiencia volvió a disminuir debido al mayor costo de coordinación entre procesos.
- Con `tile_size = 512`, el rendimiento general empeoró, con tiempos superiores a la ejecución secuencial.

3.3 Análisis de resultados

Los resultados evidencian que el paralelismo es beneficioso solo cuando el costo de procesamiento de cada tarea compensa el overhead de comunicación y sincronización. En imágenes pequeñas, el tiempo de coordinación domina sobre el procesamiento efectivo, mientras que en imágenes grandes se obtienen mejoras leves pero sostenidas.

El punto óptimo se alcanza con una cantidad moderada de workers y un tamaño de tile que permita balancear la carga. A partir de cierto umbral, agregar más procesos reduce la eficiencia debido al exceso de coordinación.

4. Conclusiones

El programa analizado logra implementar de forma adecuada los principios de la programación concurrente para tareas de tipo CPU-bound, mostrando una estructura clara y un correcto uso de los mecanismos de paralelismo en Python.

Los resultados obtenidos reflejan que el paralelismo no siempre garantiza una mejora lineal en el rendimiento, ya que factores como el tamaño del trabajo, la cantidad de procesos y el costo de comunicación influyen directamente en la eficiencia.

La experiencia permitió comprobar que las herramientas de inteligencia artificial

pueden generar código funcional y correctamente estructurado, aunque el rol del programador sigue siendo esencial para ajustar parámetros, optimizar recursos y asegurar la robustez del sistema.

En términos académicos, el ejercicio permitió afianzar conceptos teóricos sobre concurrencia y paralelismo, al mismo tiempo que evidenció la necesidad de evaluar los resultados con métricas objetivas y fundamentadas.

5. Referencias

Silberschatz, A., Galvin, P. B., & Gagne, G. (2020). Operating System Concepts (10^a ed.). Wiley.

Tanenbaum, A. S., & Bos, H. (2015). Modern Operating Systems (4^a ed.). Pearson.

IEEE Computer Society. (2021). Transactions on Parallel and Distributed Systems. IEEE Press.

Python Software Foundation. (2024). concurrent.futures — Launching parallel tasks. Recuperado de <https://docs.python.org/3/library/concurrent.futures.html>